SIMATS ENGINEERING

ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1507

COURSE OUTCOME COVERED

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct

I Bhuvankumar J (192524040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Bhuvan kumar J
192524040

Application Based Questions

1.A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

Introduction

E-commerce companies face highly unpredictable and often extreme traffic surges during seasonal sales events such as Black Friday, Diwali sales, or Amazon Prime Day. Traffic can jump from a baseline of a few thousand concurrent users to several million within minutes. Traditional, statically provisioned infrastructure cannot economically or technically handle this variability — either it is over-provisioned (wasting cost during normal periods) or under-provisioned (causing crashes during peak demand). Cloud auto-scaling combined with load balancing solves this problem by making infrastructure capacity elastic and demand-responsive.

How Auto-Scaling Works

Auto-scaling is a cloud service feature that automatically adjusts the number of compute resources (virtual machines, containers, or serverless function instances) allocated to an application based on real-time demand.

- **Metric-based triggers:** Auto-scaling groups monitor metrics such as CPU utilization, memory usage, network throughput, or request queue length. When a threshold (e.g., CPU > 70%) is crossed, new instances are automatically launched.
- **Scaling policies:** Companies can configure target-tracking scaling (maintain a metric at a target value), step scaling (add instances in increments as load increases), or scheduled scaling (pre-emptively add capacity before a known sale launch time).
- **Predictive scaling:** Machine-learning-based forecasting (e.g., AWS Predictive Scaling) analyzes historical traffic patterns from previous sales to provision capacity in advance of an anticipated spike, rather than reacting after the spike has already begun.
- **Scale-in during low demand:** Once traffic subsides, auto-scaling terminates excess instances, ensuring the company only pays for what it actually uses.

Role of Load Balancing

A load balancer sits in front of the pool of auto-scaled servers and distributes incoming client requests across them using algorithms such as round-robin, least-connections, or weighted response-time.

- **Even distribution:** No single server becomes a bottleneck, which prevents localized overload even while the overall system is under heavy load.

- **Health checks:** Load balancers continuously ping backend instances; unhealthy or unresponsive servers are automatically removed from rotation, so users are never routed to a failing server.
- **Seamless integration with auto-scaling:** As auto-scaling adds or removes instances, the load balancer automatically updates its routing table, so scaling is invisible to the end user.
- **SSL termination and traffic shaping:** Many load balancers also handle encryption/decryption and can prioritize critical traffic (e.g., checkout requests) over less critical traffic (e.g., browsing).

How This Reduces Downtime and Improves User Experience

- Prevents server crashes and '503 Service Unavailable' errors during flash sales by expanding capacity ahead of or in response to demand.
- Maintains fast page-load and checkout response times, which directly affects conversion rates — even a one-second delay can measurably reduce sales.
- Provides fault tolerance: if one availability zone or server fails, traffic is automatically rerouted to healthy servers, ensuring continuous availability.
- Improves customer trust and brand reputation, since customers experience a smooth, reliable shopping journey even during high-demand events.

Role of Monitoring Tools

Monitoring tools are the 'nervous system' that makes auto-scaling and load balancing intelligent and responsive.

- **Real-time metrics collection:** Tools like AWS CloudWatch, Azure Monitor, and Google Cloud Operations Suite continuously track CPU, memory, latency, error rates, and request counts.
- **Triggering automation:** Monitoring metrics directly feed into auto-scaling policies — without monitoring, the system has no basis on which to scale.
- **Alerting:** Teams receive real-time alerts for anomalies (e.g., a sudden traffic drop that might indicate an outage, or an unexpected cost spike), enabling rapid human intervention when needed.
- **Dashboards and post-event analysis:** After the sale, engineering teams analyze monitoring data to identify bottlenecks and fine-tune scaling thresholds for future events.

Real-World E-Commerce Example

During Flipkart's 'Big Billion Days' or Amazon's 'Great Indian Festival' sale, traffic to product and checkout pages can increase by 10–20 times the normal load within minutes of the sale opening. Auto-scaling groups automatically add hundreds of additional web and application server instances in advance (using scheduled/predictive scaling based on prior years' data), while load balancers distribute millions of concurrent user sessions across these servers and across multiple data centers/availability zones. CloudWatch-style monitoring tracks checkout latency and error rates in real time; if the payment service shows rising latency, additional instances for that specific microservice are triggered automatically. This

architecture is what allows such platforms to process millions of orders per hour without major outages, directly translating into higher completed sales and customer satisfaction.

Conclusion

Auto-scaling and load balancing together create an elastic, self-healing, and cost-efficient infrastructure that automatically matches capacity to demand. Monitoring tools provide the visibility and triggers that make this automation possible. For e-commerce companies facing seasonal spikes, this combination is essential to avoid downtime, protect revenue, and deliver a consistently fast and reliable customer experience.

2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.

Introduction

Organizations generate ever-growing volumes of critical data that must be backed up reliably and securely. Storing this data in the cloud offers a combination of durability, redundancy, security, and cost-efficiency that is difficult to replicate with traditional on-premises storage systems.

Durability and Data Redundancy

- **Replication across devices:** Cloud storage providers automatically store multiple copies of every object across different physical storage devices within a data center, protecting against individual hardware failure.
- **Multi-zone and multi-region replication:** Data can be replicated across geographically separate availability zones or even entire regions, so that a fire, flood, or power failure at one location does not result in data loss.
- **Erasure coding:** Many providers use erasure coding (splitting data into fragments with parity information) instead of simple full-copy replication, achieving high durability (often quoted as 99.999999999%, or '11 nines') with lower storage overhead than full replication.
- **Versioning:** Object versioning keeps historical copies of files, protecting against accidental deletion, corruption, or ransomware overwriting the latest version.
- **Automated integrity checks:** Cloud platforms continuously run checksums and repair processes to detect and correct silent data corruption ('bit rot') without administrator intervention.

Encryption and Access Control

- **Encryption at rest:** Backup data is encrypted using strong algorithms (e.g., AES-256) while stored on disk, so that even if physical media is stolen or improperly decommissioned, the data remains unreadable.
- **Encryption in transit:** Data moving between the organization and the cloud (and between data centers) is protected using TLS, preventing interception during upload/download or replication.
- **Key management:** Services like AWS KMS or Azure Key Vault let organizations control and rotate their own encryption keys, adding an extra layer of control (customer-managed keys) beyond provider-managed encryption.
- **Identity and Access Management (IAM):** Fine-grained, role-based access policies ensure only authorized users or systems can read, write, or delete backup data, following the principle of least privilege.
- **Multi-factor authentication and audit logging:** MFA reduces the risk of compromised credentials being used to access backups, while access logs create an audit trail for compliance and forensic investigation.
- **Immutable/WORM storage:** Write-once-read-many storage policies (e.g., S3 Object Lock) prevent backups from being altered or deleted for a set retention period — critical protection against ransomware.

Cost Advantages Over Traditional Storage

Factor	Traditional On-Premises Storage	Cloud Storage
Upfront cost	High capital expenditure for hardware	Pay-as-you-go, no upfront investment
Maintenance	Requires dedicated IT staff, cooling, physical security	Fully managed by the provider
Scalability	Limited by physical capacity; over-provisioning common	Virtually unlimited, scales on demand
Storage tiers	Single-tier, one cost regardless of access frequency	Hot/cool/archive tiers match cost to access pattern
Disaster resilience	Single location unless a second site is separately built	Built-in multi-region redundancy

Cloud storage removes hardware refresh cycles (typically needed every 3–5 years for on-premises systems), reduces staffing costs, and allows organizations to pay only for the storage tier and volume they actually use — archiving rarely accessed backups to low-cost cold storage instead of paying premium rates for all data.

Disaster Recovery Use Case

A regional bank stores nightly backups of its core banking transaction database in a cloud storage service, replicated automatically across two geographically distant regions. All backups are encrypted at rest with

customer-managed keys and protected with object-lock immutability to guard against ransomware. When the bank's primary data center is disabled by a flood, IT teams initiate a recovery process that restores the most recent backup from the secondary region. Because the data was already replicated and durable, the bank restores core operations within a few hours rather than days, minimizing financial loss, regulatory exposure, and reputational damage — an outcome that would be far costlier and slower to achieve with a single on-premises backup site.

Conclusion

Cloud storage provides durability through automated replication and erasure coding, protects sensitive data through layered encryption and strict access control, and offers substantial cost savings through elastic, tiered, pay-as-you-go pricing. These combined benefits make cloud storage a superior choice for large-scale, secure backup and disaster recovery compared to traditional storage systems.

3.A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

Introduction

When a mobile app is deployed globally, users in different continents will experience vastly different performance if all requests must travel back to a single, centrally located origin server. Content Delivery Networks (CDNs) and edge computing solve this by bringing content and computation physically closer to users, which is essential for latency-sensitive applications such as live streaming and online gaming.

How CDNs Improve Performance and Reduce Latency

- **Content caching:** CDNs store (cache) copies of static assets — images, videos, app updates, API responses — on servers, called Points of Presence (PoPs), distributed across the globe.
- **Reduced round-trip time:** Instead of every request traveling to a single origin server (which may be thousands of kilometers away), requests are served from the nearest PoP, dramatically cutting network latency.
- **Reduced origin server load:** Because most requests are served from cache, the load on the origin server drops significantly, improving reliability and reducing infrastructure costs.
- **Traffic surge absorption:** CDNs absorb sudden traffic spikes (e.g., an app going viral in one region) without overwhelming the origin infrastructure.
- **DDoS mitigation:** The distributed nature of CDNs also provides a layer of protection against distributed denial-of-service attacks, since malicious traffic is spread across many edge nodes.

Role of Edge Servers

- **Physical proximity:** Edge servers are deployed in regional or local data centers close to end users, minimizing the physical distance data must travel.

- **Edge computing:** Beyond simple caching, modern edge servers can execute lightweight logic — such as authentication checks, personalization, image resizing, or A/B testing — directly at the edge, without a round trip to the origin.
- **SSL/TLS termination:** Handling encryption/decryption at the edge reduces latency for secure connections.
- **Regional failover:** If one edge node or region experiences issues, traffic can be automatically redirected to the next nearest healthy node, maintaining uptime.

How Cloud Providers Support Global Availability

- Major cloud providers (AWS with CloudFront, Microsoft Azure with Front Door, Google Cloud with Cloud CDN) operate CDN networks with hundreds of PoPs across continents.
- They allow multi-region deployment of backend services (databases, APIs, application servers), so that if one region fails, traffic can be redirected to another functioning region.
- Global load balancing (e.g., geo-DNS routing) automatically directs each user to the nearest healthy regional deployment.
- Integrated monitoring provides visibility into regional performance, allowing providers and companies to detect and resolve regional slowdowns quickly.

Application to Real-Time Streaming or Gaming

In real-time applications, even small amounts of latency (measured in milliseconds) are highly noticeable to users, unlike loading a static webpage where a small delay is less critical.

- **Live video streaming (e.g., Twitch, Hotstar during a cricket match):** Video segments are cached and delivered from the nearest CDN edge node, allowing millions of concurrent viewers worldwide to watch with minimal buffering, and CDNs dynamically adjust video bitrate delivery based on each user's network conditions.
- **Online multiplayer gaming:** Game state synchronization between players requires extremely low latency; cloud providers deploy regional game servers so that players are matched with and connected to the nearest server cluster, reducing lag and ensuring fair, synchronized gameplay across regions.
- **Global app updates:** CDN-cached app binaries and patches allow users worldwide to download updates quickly from a nearby edge location instead of a single distant origin server.

Conclusion

CDNs and edge servers are essential architectural components for any globally deployed application. By caching content close to users and enabling computation at the edge, they drastically reduce latency and improve reliability, while cloud providers' global infrastructure ensures consistent availability across regions — capabilities that are especially critical for latency-sensitive services like live streaming and online gaming.

4.A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.

Introduction

Financial institutions handle highly sensitive customer data — account details, transaction histories, and personal identification information — and are subject to strict regulatory frameworks (such as RBI guidelines in India, PCI-DSS for card payments, GDPR in Europe, and SOX in the United States). Adopting cloud computing requires robust mechanisms for compliance, security, and data sovereignty to meet these obligations.

How Cloud Providers Support Regulatory Compliance and Auditing

- **Compliance certifications:** Major cloud providers maintain certifications against widely recognized standards (ISO 27001, SOC 1/2/3, PCI-DSS, GDPR-readiness), which financial institutions can rely on as part of their own compliance posture.
- **Compliance documentation tools:** Services such as AWS Artifact or Azure Compliance Manager give organizations on-demand access to audit reports and certification documents needed for regulatory inspections.
- **Shared responsibility model:** Providers secure the underlying infrastructure, while the financial institution is responsible for securing its applications, data, and access configurations — a clear division that supports structured compliance management.
- **Configuration and compliance auditing tools:** Services like AWS Config or Azure Policy continuously check whether cloud resources comply with defined security and regulatory rules, flagging or automatically remediating violations.

Role of Logging, Monitoring, and Encryption in Security

- **Logging:** Every API call, login attempt, and administrative action is logged (e.g., via AWS CloudTrail), creating a tamper-evident audit trail that regulators can review to verify who accessed what data and when.
- **Monitoring:** Real-time monitoring tools detect anomalies such as unusual login locations, repeated failed authentication attempts, or abnormal data transfer volumes, triggering automated alerts or account lockdowns before a breach can escalate.
- **Encryption at rest and in transit:** Sensitive financial data is encrypted using strong algorithms both when stored and when transmitted, ensuring that intercepted or stolen data remains unreadable without the proper decryption keys.
- **Tokenization:** Sensitive fields such as card numbers can be replaced with non-sensitive tokens for processing, reducing the exposure of raw sensitive data even within internal systems.

- **Continuous vulnerability scanning:** Automated scanning tools identify misconfigurations or unpatched vulnerabilities in cloud resources before they can be exploited.

Ensuring Data Sovereignty

- **Region selection:** Cloud providers allow institutions to explicitly choose the geographic region where data is stored and processed, ensuring compliance with national data localization laws (e.g., India's requirement that certain payment data be stored only within the country).
- **Data residency guarantees:** Providers offer contractual and technical guarantees that data will not be moved outside a specified jurisdiction without authorization.
- **Sovereign cloud offerings:** Some providers offer dedicated 'sovereign cloud' regions operated under local legal and operational control specifically to meet strict data sovereignty requirements.
- **Access restriction by geography:** IAM policies can restrict data access to personnel or systems located within approved jurisdictions.

Example: Sensitive Financial Transactions

Consider a bank processing UPI or card-based transactions. Transaction data is stored in a cloud region located within the bank's home country to satisfy data localization regulations. All transaction records are encrypted at rest using bank-managed encryption keys, and TLS encrypts data in transit between the payment gateway and backend systems. CloudTrail-style logging records every access to transaction records, supporting regulatory audits. Real-time monitoring analyzes transaction patterns to detect potential fraud (e.g., unusually large transfers or logins from unexpected locations), automatically flagging or blocking suspicious activity. During a regulatory audit, the bank uses the provider's compliance documentation and its own access logs to demonstrate that all controls required under RBI/PCI-DSS guidelines have been properly implemented.

Conclusion

Cloud providers offer a strong foundation for regulatory compliance through certifications, documentation, and configuration auditing tools, while logging, monitoring, and encryption form the technical backbone of financial data security. Careful region selection and access controls allow financial institutions to meet data sovereignty requirements, enabling them to adopt cloud computing without compromising regulatory obligations or customer trust.

5.A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments.
Provide an example of a microservices-based application.

Introduction

As development teams move toward microservices architectures, they need a deployment approach that is lightweight, portable, and easy to scale. Containerization, combined with orchestration platforms like Kubernetes, has become the standard solution, offering significant advantages over traditional virtual machine (VM)-based deployment.

Containers vs. Virtual Machines

Aspect	Virtual Machines	Containers
Virtualization level	Hardware-level (each VM has a full guest OS)	OS-level (containers share the host kernel)
Resource usage	High (each VM duplicates OS overhead)	Low (shared kernel, minimal overhead)
Startup time	Minutes	Seconds or less
Density per host	Lower (fewer VMs fit on one server)	Higher (many more containers fit on one server)
Isolation	Strong (separate kernel/OS per VM)	Process-level isolation (slightly less strict, but generally sufficient)
Portability	Less portable; tied to hypervisor/image format	Highly portable across any environment supporting the container runtime

Because containers package only the application and its dependencies (not a full operating system), they consume far less CPU, memory, and storage than VMs, and can be started, stopped, or replicated almost instantly. This makes containers ideal for microservices, where many small, independent services need to be deployed and scaled frequently.

How Kubernetes Orchestration Simplifies Deployment and Scaling

- **Automated scheduling:** Kubernetes automatically decides which physical or virtual node should run each container based on available resources, without manual intervention.
- **Auto-scaling:** The Horizontal Pod Autoscaler automatically increases or decreases the number of running container instances (pods) based on real-time CPU, memory, or custom metrics.
- **Self-healing:** If a container crashes or a node fails, Kubernetes automatically restarts or reschedules the affected containers onto healthy nodes, minimizing downtime without human intervention.

- **Service discovery and internal load balancing:** Kubernetes automatically assigns stable network identities to services and load-balances traffic across all healthy replicas of a given microservice.
- **Rolling updates and rollbacks:** New versions of a microservice can be deployed gradually with zero downtime, and Kubernetes can automatically roll back to the previous version if errors are detected.
- **Declarative configuration:** Teams define the desired state of their application (number of replicas, resource limits, networking rules) in configuration files, and Kubernetes continuously works to maintain that state.

Portability Benefits Across Cloud Environments

- Containers package the application code, runtime, libraries, and configuration together, ensuring identical behavior whether run on a developer's laptop, an on-premises server, or any public cloud.
- This eliminates the classic 'it works on my machine' problem caused by environment inconsistencies.
- Because Kubernetes is supported by all major cloud providers (Amazon EKS, Google GKE, Azure AKS) as well as on-premises and hybrid setups, applications can be moved between clouds with minimal changes, avoiding vendor lock-in.
- Teams can adopt a multi-cloud or hybrid-cloud strategy, running workloads wherever it is most cost-effective or compliant, while using the same deployment tooling and processes everywhere.

Example: Microservices-Based Application

Consider an e-commerce platform decomposed into independent microservices: user authentication, product catalog, shopping cart, payment processing, order management, and product recommendations. Each microservice is packaged as its own container image with only the dependencies it needs. Kubernetes orchestrates all these containers: during a sale, the Horizontal Pod Autoscaler increases the number of payment-service and cart-service replicas independently of the less-loaded recommendation service, based on real-time demand for each. If a node hosting several containers fails, Kubernetes automatically reschedules those containers onto healthy nodes with no manual intervention. Development teams can update the recommendation engine and deploy it via a rolling update without affecting or redeploying any other microservice, and the entire system can be migrated from one cloud provider to another with minimal reconfiguration because it is built on portable container images and standard Kubernetes manifests.

Conclusion

Containers offer major performance and resource-efficiency advantages over virtual machines by sharing the host OS kernel and eliminating redundant overhead. Kubernetes orchestration automates scheduling, scaling, healing, and updates, drastically simplifying the operational burden of running many microservices. Combined with strong portability across cloud environments, containerization and orchestration form the ideal foundation for modern, scalable, cloud-native microservices architectures.

Application Based Questions Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity